



北京邮电大学

BEDING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第八章 极化码

李丽香, 彭海朋

北京邮电大学网络空间安全学院

网络与交换技术国家重点实验室





北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

极化 (Polar) 码起源

极化码 (Polar Code) 是由土耳其 Bilkent 大学的 Erdal Arkan 教授于 2008 年发明的一种信道编码方法。Polar 码是目前唯一的一种能够达到香农极限 (Shannon Limit) 的编码方法, 而比较其他编码方法只是接近香农极限。也是目前国际移动通信标准化组织认定的 5G 场景控制信道的编码方案。



Page 265



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

极化码优缺点

极化码优点:

1、是目前唯一的香农信道容量可达的编码方式

2、有坚实的理论基础

3、编解码复杂度低

4、精细的码率调整机制 (信息块长度可以一比特一比特的递减)

5、极化码的递归特性易于通过硬件实现 (母码长度为 N 的 Polar 码可以用两个母码长度为 N/2 的 Polar 码实现)

极化码缺点:

1、汉明距离较小, 在短码情况下影响解码性能, 一定程度上可以通过选择合适的冻结比特位置进行规避。适合长码编码。

2、SC 译码延迟较大。

Page 266



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

国外研究现状

2009

日本京都大学的教授使用密度递进的方法研究了 Polar 码的性能。

2013

英国德州大学的 L. S. 研究了在衰落信道下的分段 Polar 码, 其编码方案达到了衰落 DM-CC 下的性能。

2014

加拿大和 P. Guo 研究了在普通 CPU 上实现的 Polar 码软件解码算法, 其吞吐量超过了 200Mbps。

2017

在美国 Spokane 的 W. P. RAN WG10B 会议上, 李丽香和特翻特模块的 Polar 码得以确定在 5G 中使用

2018

美国 RAN 的 W. P. TSG RAN WG10B 会议上, Polar 码被确定为第五代移动通信 5G 的增强移动宽带 eMBB 信道控制信道的编码方案。

2019

在 5G 的 W. P. RAN 会议上, 下一代新的无线接入网 (NR, 5G, 5G-NR) 的研究项目的立项通过, Polar 码在短码长, 低码率上的性能优势。

Page 267



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

国内研究现状

- 在国内, 北京邮电大学是最早研究 Polar 码的机构之一。牛凯教授的团队研究了 Polar 码的编码和解码算法等。
- 西安电子科技大学李颖教授的团队研究了 Polar 码的高斯近似的码构造方法和解码算法。
- 哈尔滨工业大学王学东教授的团队研究了 Polar 码的编码与译码算法。

Page 268



北京邮电大学

BEIJING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

极化 (Polar) 码起源

截止频率越高, 系统响应越快
因此 Polar 码的最初设计的出发点是为了提升截止频率

在随机编码和最大似然条件下, 信道截止速率 R_0 决定了相应的码块的错误概率:

$$P_e = 2^{-N R_0}$$

其中, N 为码块长度。实际信道的传输速率 R 小于 R_0 。使用随机编码和 ML 编码时, 信道平均错误概率:

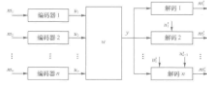
$$\bar{P}_e = 2^{-N (R_0 - R)}$$

Page 269

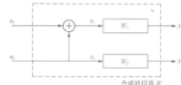
1

因此，在2004-2006年，Arikan 教授尝试使用信道合并与分离的方法来提升信道截止速率。

对于一个离散无记忆的信道 W ，分离出两个相关的子信道 W_1 和 W_2 ，有 $C(W_1)+C(W_2)\leq C(W)$ ，同时有 $R_0(W_1)+R_0(W_2)>R_0(W)$ 。其中 C 是信道容量， R_0 是信道截止速率，即两个子信道信道容量不超过原本的总信道容量，但分离后总信道截止速率提高了。



信道分离示意图



信道合并示意图

Page 270

对于Polar码基本原理可以归结为三点：信道合并、信道分离、信道极化。其中信道合并和信道极化是在编码时完成的；信道分离是在解码时完成。

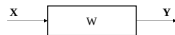
信道选择：

对于编码方案通常根据某种信道设计，Polar码主要考虑以下信道进行分析：

二进制离散无记忆信道 (B-DMC) $\begin{cases} \text{二进制删除信道 (BEC)} \\ \text{二进制对称信道 (BSC)} \end{cases}$

Page 271

信道 $W: X \rightarrow Y$ 。输入变量 X 只能取 $\{0,1\}$ 中的值，输出变量 Y 取任意值，信道迁移概率为 $P=W(y|x)$ ， $x \in X$ ， $y \in Y$ 。



信道示意

对于对称信道 (BEC/BSC)，当输入在 $\{0,1\}$ 概率时，其信道容量定义为输入输出的相互信息。

$$C(W)=I(X;Y)=\sum_{y \in Y} \sum_{x \in X} \frac{1}{2} W(y|x) \log \left(\frac{W(y|x)}{\frac{1}{2} W(y|0) + \frac{1}{2} W(y|1)} \right)$$

Page 272

信道合并（编码时）：

将两个BEC信道 W_1 和 W_2 合并成一个信道 W ，其中，“+”为模2加。由于 u_1 和 u_2 都取自 $\{0,1\}$ 且独立同分布，那么信道容量为：

$$C(W)=I(U;Y)$$

$$\text{其中，} U=[u_1, u_2], Y=[y_1, y_2]$$

$$C(W)=I(X;Y)$$

$$\text{其中，} X=[x_1, x_2]$$

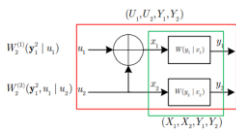
$$C(W)=I(x_1, y_1) + I(x_2, y_2) = C(W_1) + C(W_2) \\ \approx 2 \times C(W_1) = 2 \times C(W_2)$$

也就是说，信道合并并保持总信道容量守恒。若 N 个信道合并，则

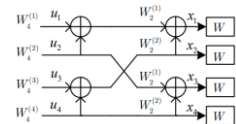
$$C(W) \approx N \cdot C(W_1)$$

即，合成信道的总容量是各个子信道容量之和，容量守恒。

Page 273

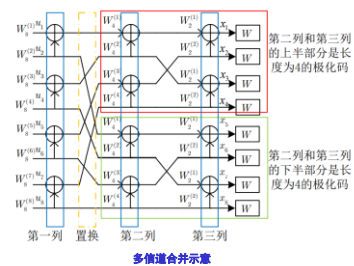


双信道合并示意



4个BEC信道合并示意

Page 274



多信道合并示意

Page 275



极化码基本原理

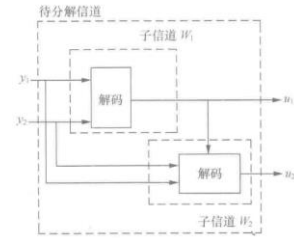
信道分离（解码时）：

信道分离是将混有多个输入比特信息的接收信息 $y_1^N = (y_1, y_2, y_3, \dots, y_N)$ 分解成独立的多个输入比特信息 $u_1^N = (u_1, u_2, u_3, \dots, u_N)$ 。考虑针对G2 Polar码编码而成的信道解码过程。首先解调器从 $y^N = y^2 = (y_1, y_2)$ 分解出第一个子信道 u_1 。之后再从 (u_1, y_1, y_2) 中分解出第二个子信道 u_2 ，从而完成了 $y^N \rightarrow u^N$ ， $N=2$ 的信道分解过程。信道分离过程也是容量守恒的。

Page 276



极化码基本原理



信道分离示意图

Page 277



极化码基本原理

信道极化（编码时）：

信道极化是指随着Polar码母码长度 $N=2^n$ 的增大，子信道（Polar码输入比特序号）的信道容量会越来越明显的趋向两个极端：无噪信道（信道容量为1）和纯噪信道（信道容量为0）。

信道极化（编码时）：

假设使用BEC信道，且删除概率为 $P = 1/2$ ，那么随着Polar码母码长增大，子信道趋近于两个极端，当母码长度 $N \geq 512$ 时，极化现象体现较为充分（大于80%的子信道得到极化），利用这个极化现象，可以选择高容量子信道去传输数据，而把其他子信道设置为已知比特（冻结比特，如0）。

Page 278



极化码基本原理

若使用以2为底的对数，则 $0 \leq C(W) \leq 1$ 。当 $C(W)=1$ 时，称 W 为无噪信道（极好）；当 $C(W)=0$ 时，称 W 为纯噪信道（无用）。信道极化的目的就是大多数常规信道转化为这两种极端情况的信道，上述信道容量计算复杂，可采用巴氏参数的最大后验概率（MAP）判决错误概率的上界，进而进行信道容量的范围考察。巴氏参数定义为 $Z(W)$ ：

$$Z(W) = \sum_{y \in Y} \sqrt{W(y|0)W(y|1)}$$

主要通过信道容量分析极化码。

$$C(W) \geq \log \frac{2}{1+Z(W)}$$

$$C(W) \leq \sqrt{1-Z(W)} \cdot Z(W)$$

Page 279



极化码基本原理

因此，针对二进制删除信道（BEC），信道容量 $I(w_N^{(i)})$ 数据可以通过递归关系计算出来。

$$I(w_N^{(2i-1)}) = I(w_{N/2}^{(i)})^2$$

$$I(w_N^{(2i)}) = 2I(w_{N/2}^{(i)}) - I(w_{N/2}^{(i)})^2$$

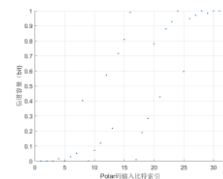
其中 $I(w_1^{(1)}) = 1 - \epsilon$ ，删除概率 $\epsilon = 0.5$ 。

Page 280



极化码基本原理

信道极化的统计图显示了 $I(w^{(i)})$ 对于小的指数 i 趋向接近于“0”，对于大的指数 i 趋向接近于“1”。然而，对于中间范围的指数 i ， $I(w_N^{(i)})$ 显示了不规则的特性。图中的所有点是呈现中心对称的，这也是极化码的信道极化现象所具有的特性。



Polar码信道容量 (N=32)

码长 $N=2^5$ 的信道极化现象

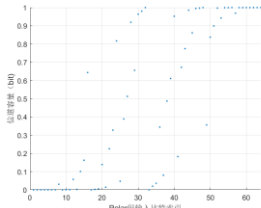
Page 281



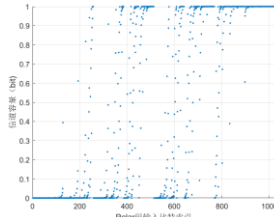
极化码基本原理

信道极化的统计图

随着子信道越来越多，信道容量就会越来越趋近于两端。



Polar码信道容量 (N=64)
码长N=2^n的信道极化现象



码长N=2^10的信道极化现象

Page 282



Polar码的构造

Polar码构造是指为Polar码能够良好工作设计的框架，其中包括编码矩阵生成、序列、速率匹配、交织



Polar码构造

Page 283



Polar码的构造—编码矩阵生成

Polar码的编码矩阵是指在进行极化编码时用来计算编码字的矩阵（ $C=U \cdot G$ 中的G）编码矩阵通常是事先存储好而不需要动态改变的。一个母码长度为 $N = 2^n$ 的极化码，其编码矩阵可由：

$$\text{Arikan核} F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

计算出来 $G_N = F^{\otimes n}$ ，其中 $\otimes$ 表示张量积。

$$\text{例如: } G_2 = F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad G_4 = F^{\otimes 2} = F \otimes F = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}.$$

Page 284



Polar码的构造—码序列

Polar码序列也称为“好信道选择”。用于指示Polar码编码前的比特选择顺序。Polar码可以按照信道的可靠度从低到高排列（默认用3GPP方式），也可以按照可靠度从高到低排列。

通常使用行权重（RW）序列，对于生成矩阵G的每一行，计算出所有元素的总和得到各行的行权重，然后对行权重由低到高排序（越大可靠度越高），则各行权重对应序号为所述RW序列。

Page 285



Polar码的构造—码序列

例如，对于 G_8 Polar码，其生成矩阵为：

$$G_8 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

各行的权重分别为1、2、2、4、2、4、4、8那么 G_8 的Polar码的（RW）序列为{0, 1, 2, 4, 3, 5, 6, 7}。如果有两行权重相同，则行号大的排在后面（也可排在前面）

Page 286



Polar码的构造—码序列特性

1、在线计算

不用存储数据或只需存储少量数据即可，但是需要消耗计算量，若计算量较大会产生时延。

2、嵌套性

在母码长度为N的序列中，删除大于或等于N/2的元素而保持剩下的元素顺序，得到母码长度为N/2的序列，则称母码长度为N/2的序列是嵌套在母码长度为N的序列中的。

嵌套的好处在于不需要重新计算，不需要重新排序，只需一个查找表。

3、对称性

在母码长度为N的序列中，前后N/2是元素关于母码长度对称，对称性可以减少存储量。

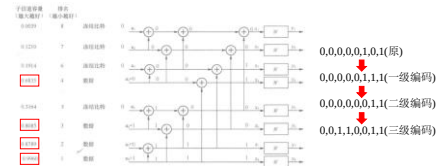
Page 287

Polar码编码需选择合适的承载数据信道和放置其他信道（冻结比特）的子信道，然后进行逻辑运算的过程。通常选用承载数据的信道应选用信道容量高或可靠度高的信道。

——根据行权重进行选取（**码序列选择**）

Page 288

例1：选用N=8、R=1/2的编码，则应选用 u_4, u_6, u_7, u_8 （0,1,0,1）来进行编码。由于要发射的比特数不一定刚好是编码之后的比特数（如要发射12个比特），因此，在编码结束后，还要进行比特选择过程（后续讲解）。



Page 289

例2：（计算）设码长N=8，消息比特数k=4。（部分教材极化码用c表示，也可用x表示）

需要的公式：

$$\begin{aligned} c_1^N &= u_1^N G_N \\ G_N &= B_N F^{\otimes n} \\ F^{\otimes n} &= F \otimes F^{\otimes (n-1)} \\ F &= \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \\ B_N &= R_N (I_2 \otimes B_{N/2}) \\ B_2 &= I_2 \end{aligned}$$

Page 290

1、假设得到的码序列选择出的四个子信道序号为“4,6,7,8”记作

$$A = \{4, 6, 7, 8\}$$

固定比特序号集合为

$$A^c = \{1, 2, 3, 5\}$$

$$\text{得到 } u_A^8 = [0, 0, 0, 1, 0, 1, 1, 1]$$

2、构造矩阵，根据公式 $B_N = R_N (I_2 \otimes B_{N/2})$ 计算 B_N

首先 $B_2 = I_2$

$$B_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$I_2 \otimes B_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Page 291

R_4 由 I_4 变换得来，先排 I_4 的奇数列，再排 I_4 的偶数列：

$$\begin{aligned} R_4 &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \xrightarrow{I_4} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ B_4 &= R_4 (I_2 \otimes B_2) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ I_2 \otimes B_4 &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

Page 292

R_8 由 I_8 变换得来，先排 I_8 的奇数列，再排 I_8 的偶数列：

$$\begin{aligned} R_8 &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \xrightarrow{I_8} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \\ B_8 &= R_8 (I_2 \otimes B_4) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \\ I_2 \otimes B_8 &= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix} \end{aligned}$$

Page 293

3、求F的n次克罗内克积 $F^{\otimes n} = F \otimes F \otimes \dots \otimes F$ (n-1)

$$F^{\otimes 1} = F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

$$\begin{cases} F^{\otimes 3} = F \otimes F^{\otimes 2} \\ F^{\otimes 2} = F \otimes F^{\otimes 1} \end{cases}$$

$$F^{\otimes 2} = F \otimes F^{\otimes 1} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \otimes \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} F^{\otimes 1} & 0 \\ 0 & F^{\otimes 1} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$F^{\otimes 3} = F \otimes F^{\otimes 2} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \otimes \begin{bmatrix} F^{\otimes 2} & 0 \\ 0 & F^{\otimes 2} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Page 294

4、求生成矩阵 $G_N = B_N F^{\otimes n}$

$$G_8 = B_8 F^{\otimes 3} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Page 295

5、生成极化码 $c_1^N = u_1^N G_N$

$$c_1^8 = u_1^8 G_8 = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Page 296

速率匹配是把Polar码编码后的比特数量 $N = 2^n$ 通过调整（增加或减少比特数量），适配到对应的物理资源（所承载的比特数量）上。

速率匹配包括两个操作，一是**交织**，二是按照一定方式**选取比特**。

速率匹配需要几个参数：

K: （分段后）编码前的信息比特长度。

M: 对应资源所承载的比特数。

$R_{\min}$: 1/8（最小码率）

$N_{\max}$: 最大码母长度。下行方向为512，上行方向是1024。

$N_{\min}$: 最小码母长度。固定为32。

N_{dm} : 大于或等于M并且是2的幂的数。

N_m : 另一个候选的码母长度。若 $M \leq (9/8) \times N_{dm}/2$ 且 $K/M < (9/16)$

则 $N_m = N_{dm}/2$; 否则, $N_m = N_{dm}$ 。

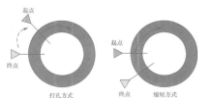
Page 297

速率匹配（选取比特）过程：

第一步：将编码之后的比特写入循环缓冲区。 **第四步**：若 $M < N$ ，并且码率 $K/M > (7/16)$ ，使用缩短

第二步：若 $M = N$ ，则没有操作。 方式。即从第一个比特开始选取，直到第M个比特。

第三步：若 $M < N$ ，并且码率 $K/M \leq (7/16)$ ，使用打孔方式。即从第 $N-M+1$ 个比特开始选取，直到最后一个比特。 **第五步**：若 $M > N$ ，则使用重复方式。除去要取出N比特之外，还需要取第一个到第 $\text{mod}[(M-N), N]$ 个比特（对M-N取N的模）。

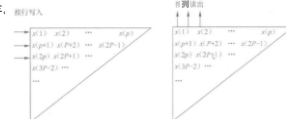


Page 298

交织是指将信息块中的比特顺序打乱，使前后比特之间尽可能减少关联性。有助于对抗突发干扰。本部分讲述后交织（交织规模较大），对时延影响较大。

一、等腰直角三角形交织

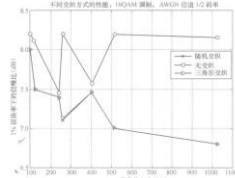
设直角边长P（3GPP规定交织器最大8192bit，即 $P \times P/2 \leq 8192$, $P \leq 128$ ），M为速率匹配后的比特数，边长P满足 $P \times (P+1)/2 \geq M$ 。需按行写入，按列读出。如果 $P \times (P+1)/2 > M$ ，需要在最后添加哑元（NULL），读出时碰到NULL则丢弃。



Page 299

等腰三角形交织器重要性质：

- 交织之前连续排列的数据，在交织之后它们的距离变成P-1,P-2,P-3...它们之间的距离不再相等。
- 每一列的长度不同，从而使得每列的转置模式都不同。

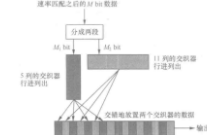


等腰三角形交织器性能

Page 300

二、双矩形交织

如图所示双矩形交织（并行矩形交织）中，首先将待交织的M bit数据分成两个部分： $M_1 = 1 + \text{ceil}(M/2)$ bit和 $M_2 = M - M_1$ bit。其次，把 M_1 bit按行写入深度为5的矩形交织器；把 M_2 bit按行写入深度为11的矩形交织器；必要时，填入NULL。然后，从这两个矩形交织器中按列读出数据。如果有NULL，则丢弃它。最后将这些数据交叠地写入一个M bit的交织器。



双矩形交织器示意

Page 301

Polar码解码方式与编码过程一样，即对接收到的信息y估计成c，然后对c进行Polar码编码即可得到原始的原始的发送信息u。

由于编码过程： $x = u \cdot G$
 等式两边同时右乘 G^{-1} 得到： $u = x \cdot G^{-1}$ （ G^{-1} 是G的逆矩阵）
 由于 $G^{-1} = (F^{\otimes n})^{-1} = (F^{-1})^{\otimes n} = G$ ，
 所以， $u = x \cdot G^{-1} = x \cdot G$ 。
 其中F为 $F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ 。

Page 302

Polar码解码经常使用串行消去（SC）解码方法。

串行消去：

- ① 将比特索引号大的一半（N/2个数据）当做随机噪声，根据接收到的数据去解码比索引号小的一半（N/2个数据）；
- ② 把解出来的比特索引号小的一半（N/2个数据）当做已知比特，根据接收到的数据去解码比索引号大的一半（N/2个数据）。

Page 303

举例：假设从比特到符号的BPSK映射规则为 $y=2x-1$ （比特“0”映射为“-1”，比特为“1”映射为“+1”），还假设接收到的数据为 y_1, y_2, y_3, y_4 。那么，对这些接收到的数据转换成对数似然比（LLR）

$$x = LLR(y) = \ln \frac{W(y|0)}{W(y|1)}$$

其中， $W(y|0)$ 表示发射端发射的是比特“0”，而接收端收到的是y的概率， $W(y|1)$ 表示发射端发射的是比特“1”， $\ln(*)$ 表示自然对数的底e为底的数。

Page 304

举例：在现实计算时， $W(y|1)$ 可用下式计算：

$$W(y|1) = \frac{1}{1 + e^{-2y/\delta^2}}$$

其中， δ^2 为噪声方差（可通过对接收到的数据y统计得到，一半设置 $\delta^2=1$ ）

对于 $W(y|0)$ ，可用 $W(y|0)=1-W(y|1)$ 计算。

Page 305

N=4解码步骤示意:

第一步：解码器把来自下半部分 (x_3, x_4) 的信息当做噪声，根据上半部分 (x_1, x_2) 计算出临时变量 t_1, t_2 。在这里，“当做噪声”的含义是从编码（或数据发射）角度看，有

$$x_1 = t_1 + n_1$$

$$x_2 = t_2 + n_2$$

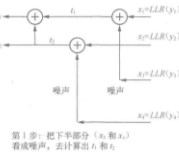
其中 t_1, t_2 为有用数据， n_1, n_2 为噪声，因此在解码时为消除噪声影响需要

$$t_1 = x_1 - n_1$$

$$t_2 = x_2 - n_2$$

这里的“-”不是简单的减法，二是/运算（盒子加，Box-plus），如下。

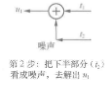
$$t_1 = x_1 - n_1 = f(x_1, n_1) = x_1 \boxplus n_1 = \ln \frac{1 + e^{x_1} e^{n_1}}{e^{x_1} + e^{n_1}}$$



第1步：把下半部分 (x_3 和 x_4) 看成噪声，去计算出 t_1 和 t_2

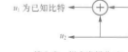
Page 306

第二步：解码器把下半部分（临时变量 t_2 ）看成噪声，求解出 u_1 。



第2步：把下半部分 (t_2) 看成噪声，去解出 u_1

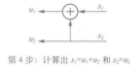
第三步：把上半部分 (u_1) 当做已知比特，去解出 u_2 。此时应根据 u_1 的取值 (0,1) 来对临时变量进行 t_1, t_2 的操作，然后再译出 u_2 。



第3步：把上半部分 (u_1) 当作已知比特，去解出 u_2

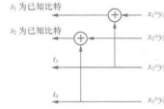
Page 307

第四步：计算出另一组临时变量 $s_1 = u_1 + u_2$ （这里指模2加“ $\boxplus$ ”）和 $s_1 = u_2$



第4步：计算出 $s_1 = u_1 \boxplus u_2$ 和 $s_2 = u_2$

第五步：把上半部分（临时变量 s_1 和 s_2 ）当做已知比特，去计算又一组临时变量 t_3, t_4



第5步：把上半部分 (s_1 和 s_2) 当作已知比特，去计算 t_3 和 t_4

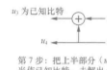
Page 308

第六步：与第二步类似，把下半部分（临时变量 t_2 ）看成噪声，求解出 u_3 。



第6步：把下半部分 (t_2) 看成噪声，去解出 u_3

第七步：与第三步类似，把上半部分 (u_3) 当做已知比特，去解出 u_4 。



第7步：把上半部分 (u_3) 当作已知比特，去解出 u_4

Page 309

例3：假设接收到数据为 $y_1 = -1.61$ 、 $y_2 = -0.06$ 、 $y_3 = -1.89$ 、 $y_4 = 1.69$ ，因为 $x = \text{LLR}(y)$ ，所以 $x_1 = 3.22$ 、 $x_2 = 0.12$ 、 $x_3 = 3.78$ 、 $x_4 = -3.38$ 。计算解码结果。

第一步：将 x_3, x_4 当做噪声，根据上半部分 x_1, x_2 计算临时变量 t_1, t_2 。

$$t_1 = f(x_1, x_3) = x_1 \boxplus x_3 = 2.77$$

$$t_2 = f(x_2, x_4) = x_2 \boxplus x_4 = -0.11$$

Page 310

第二步：解码器把下半部分（临时变量 t_2 ）看成噪声，求解出 u_1 。那么，

$$z_1 = f(t_1, t_2) = t_1 \boxplus t_2 = -0.09$$

对 z_1 进行判决（若是冻结比特，则无需判决，直接输出冻结比特值），得到 $u_1 = 1$ （这一比特解码错误；从仿真程序看，SNR=0dB，下同；考虑码长 N=4 不够大，极化现象不明显，且码率 R=1，有错误正常）

$$\text{判别条件: } u_i = \begin{cases} 0, & \text{如果 } \text{LLR}_i \geq 0 \\ 1, & \text{其他} \end{cases}$$

Page 311

解码示例

第三步: 把上半部分 (u_1) 当做已知比特, 去解出 u_2 。此时应根据 u_1 的取值 (0,1) 来对临时变量进行 t_1, t_2 的操作, 然后再译出 u_2 。

根据编码规则, 如果 $u_1=0$, 则编码之后有 $t_1 = t_2$, 那么对 u_2 进行解码时, $z_2 = t_1 + t_2$, 对 z_2 进行判决得到 u_2 ;

若 $u_1=1$ ，则在编码之后有 $t_1 = 1 - t_2$ （经BPSK后 $t_1 + t_2 = 0$ ），那么对 u_2 解码时，有 $z_2 = t_2 - t_1$ ，对 z_2 进行判决，得到 $u_2=1$ （解码正确）

Page 312



解码示例

第四步：计算出另一组临时变量 $s_1 = u_1 \oplus u_2$ 和 $s_2 = u_2$ 。根据上面第2~3步结果得到 $s_1=0, s_2=1$ 。

第五步:把上半部分(临时变量 s_1 和 s_2)当做已知比特,去计算又一组临时变量 t_3, t_4 。类似于第三步,有

$$t_3 = x_3 + (1-2 \times s_1) - x_1 = 3.78 + (1-2 \times 0) \times 3.22 = 7.00$$

$$t_4 = x_4 + (1-2 \times s_2) - x_2 = -3.38 + (1-2 \times 1) \times 0.12 = -3.50$$

Page 313



解码示例

第六步：与第二步类似，把下半部分（临时变量 t_2 ）看成噪声，求解出 u_3 。

$$z_3 = f(t_3, t_4) = t_3 \boxplus t_4 = -3.47$$

对 z_3 进行判决, 得到 $u_3=1$ (解码错误)

第七步：与第三步类似，把上半部分 (u_3) 当做已知比特，去解出 u_4 。

$$z_4 = t_4 + (1-2 \times u_3) \times t_3 = -3.50 + (1-2 \times 1) \times 7.00 = -10.50$$

对 z_4 进行判决, 得到 $u_4=1$ (解码正确)

Page 314



解码示例

小结:

上述解码方法是串行进行的(先解 u_1 , 然后解 u_2 , 再解 u_3 , 最后解 u_4)并且和解码后面的比特(如 u_2)时, 充分考虑了前面比特(u_1)的影响, 从而逐步消除前面比特的影响, 故称之为串行消除算法。

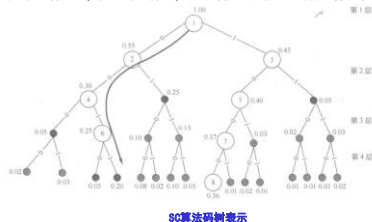
当母码长度更大的时候，需要执行多次上面这些步骤。

Page 315



解码算法

普通SC算法解码方式主要通过后验概率进行判决，总是通过最大后验概率的路径进行分析，但通过SC算法，有时可能会走错。（码树进行表示）



Page 314



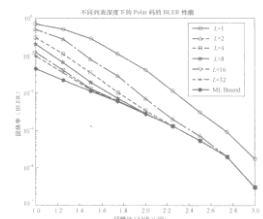
解码算法

第二种方式:SC-L算法

SC-L算法是在SC算法的基础上,通过保存L条具有最大后验概率的路径来进行下一层的解码,算法在最后选取一条具有最大后验概率的路径。即相对SC算法,SC-L算法增加了路径保存和回退功能。

当列表深度 $L=1$ 时, SC-L算法等同于SC算法;

当列表 $L = \min(2^N, 2^{\text{Length}(A)})$ 时, SC-L 算法就变成了 ML 算法。Length(A) 为信息集的长度。当列表长度 $L=32$ 时, SC-L 算法已经与 ML 非常接近了。



Page 317

